LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 9 – STATIC CLASS DAN COLLECTION



NAMA: HUMAIRAH NURHAMIDAH

NIM: 103132400038

KELAS: PS1DS-05-REG-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Static adalah anggota milik kelas, bukan milik objek, sehingga bisa diakses tanpa membuat instance. *Static variable* (variabel kelas) dipakai bersama oleh semua objek, jadi perubahan nilainya dari satu objek langsung terlihat di objek lain. Variabel lokal tidak bisa dideklarasikan sebagai static. *Static method* dapat dipanggil langsung lewat nama kelas, tetapi hanya bisa memanggil method static lain dan mengakses variabel static, tidak bisa mengakses variabel atau method non-static. Dalam konteks tugas ini, static variable cocok dipakai sebagai penghitung jumlah objek yang sudah dibuat.

Collection adalah objek yang menampung banyak objek sekaligus. Collection didefinisikan sebagai interface dan memiliki turunan seperti List (ArrayList, LinkedList, Vector, Stack), Queue (PriorityQueue, Deque), dan Set (HashSet, LinkedHashSet, TreeSet). Beberapa method dasarnya adalah add(), remove(), addAll(), removeAll(), retainAll(), dan contains(). Dengan *generics* (misalnya List<String>), tipe data yang boleh ditampung dibatasi, sehingga lebih aman dan tidak perlu casting. Elemen collection dapat ditelusuri dengan for biasa, for-each, iterator, lambda expression, maupun method reference.

Sorting pada collection bisa dilakukan dengan dua cara. Pertama, interface **Comparable**, yaitu kelas meng-override compareTo() untuk menentukan urutan default pada satu properti. Method ini mengembalikan nilai negatif, nol, atau positif tergantung posisi objek terhadap pembandingnya. Kedua, interface **Comparator**, yaitu membuat kelas terpisah yang meng-override compare() untuk urutan alternatif berdasarkan properti lain. Pengurutan dijalankan dengan Collections.sort(list) atau Collections.sort(list, comparator).

Filtering pada collection dapat dilakukan secara konvensional dengan perulangan dan if, atau lebih ringkas dengan ekspresi lambda melalui Stream API. Method filter() menyeleksi elemen sesuai kondisi, collect(Collectors.toList()) mengumpulkan hasilnya ke list baru, dan findFirst().orElse(null) dipakai untuk mencari satu elemen yang cocok dengan hasil null jika tidak ditemukan.

B. Guided

1. Guided01

[Share.java]

```
package guided01;


public class Share {

    private int privateInt;

    private static int staticInt;

    public Share(int pr, int si){

        privateInt = pr;

        staticInt = si;

    }


    public String toString(){

        return privateInt + " " + staticInt;

    }

}
```

[Driver.java]

```
package guided01;

public class Driver{
    public static void main(String args[]){
        Share s1 = new Share(4,4);
        System.out.println(s1.toString());

        Share s2 = new Share(8,2);
        System.out.println(s1.toString());
        System.out.println(s2.toString());

        Share s3 = new Share(6,22);
        System.out.println(s1.toString());
        System.out.println(s2.toString());
        System.out.println(s3.toString());
```

```
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ ProjectModul09 ---
4 4
4 2
8 2
4 22
8 22
6 22
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

2. Guided02

[CounterMachine.java]

```
package guided02;

public class CounterMachine {
    static int counter = 0;

    public static void count(){
        counter++;}

}
```

[Driver.java]

```
package guided02;

public class Driver{
    public static void main(String args[]){
        for(int i = 0; i< 10; i++){
            if( i % 2 == 0){
                CounterMachine.count();
            }
        }

        System.out.println(CounterMachine.counter);
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ ProjectModul09 ---
5
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

3. Iteration

[MainIteration.java]

```
package Iteration;

import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

public class MainIteration {
    public static void main(String[] args) {
        // 1. Inisialisasi List bertipe String
        List<String> listBuah = new ArrayList<>();

        // 2. Menambah elemen ke dalam List
        listBuah.add("Apel");
        listBuah.add("Mangga");
        listBuah.add("Jeruk");
        listBuah.add("Pisang");

        // 3. Operasi utilitas List
        System.out.println("Total elemen: " + listBuah.size());
        System.out.println("Apakah ada 'Mangga'? " + listBuah.contains("Mangga"));

        // Menghapus data
        listBuah.remove("Pisang");

        // --- TEKNIK 1: For-Loop Tradisional (Indeks) ---
        System.out.println("\n--- 1. Normal For-Loop (Indeks) ---");
        for (int i = 0; i < listBuah.size(); i++) {
            System.out.println("Index " + i + ": " + listBuah.get(i));
        }

        // --- TEKNIK 2: Enhanced For-Loop (For-Each) ---
        System.out.println("\n--- 2. Enhanced For-Loop (For-each) ---");
```

```
        for (String buah : listBuah) {
            System.out.println(buah);
        }

        // --- TEKNIK 3: Iterator Interface ---
        System.out.println("\n--- 3. Menggunakan Iterator ---");
        Iterator<String> itr = listBuah.iterator();
        while (itr.hasNext()) {
            System.out.println(itr.next());
        }

        // --- TEKNIK 4: Lambda Expression (Java 8+) ---
        System.out.println("\n--- 4. Lambda Expression ---");
        listBuah.forEach(item -> System.out.println(item));

        // --- TEKNIK 5: Method Reference (Java 8+) ---
        System.out.println("\n--- 5. Method Reference ---");
        listBuah.forEach(System.out::println);
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ ProjectModul09 ---
Total elemen: 4
Apakah ada 'Mangga'? true

--- 1. Normal For-Loop (Indeks) ---
Index 0: Apel
Index 1: Mangga
Index 2: Jeruk

--- 2. Enhanced For-Loop (For-each) ---
Apel
Mangga
Jeruk

--- 3. Menggunakan Iterator ---
Apel
Mangga
Jeruk

--- 4. Lambda Expression ---
Apel
Mangga
Jeruk

--- 5. Method Reference ---
Apel
Mangga
Jeruk
------------------------------------------------------------------------
BUILD SUCCESS
```

4. Sorting

[Employee.java]

```
package sorting;

public class Employee implements Comparable<Employee> {
    private String name;
    private int salary;

    public Employee(String name, int salary) {
        this.name = name;
        this.salary = salary;
```

```
    }

    public String getName() {
        return name;
    }

    public int getSalary() {
        return salary;
    }

    // Default sorting: Berdasarkan nama (A-Z)
    @Override
    public int compareTo(Employee other) {
        return this.name.compareTo(other.name);
    }

    @Override
    public String toString() {
        return String.format("Employee[name=%-6s, salary=%d]", name, salary);
    }
}
```

[SalaryComparator.java]

```
package sorting;

import java.util.Comparator;

public class SalaryComparator implements Comparator<Employee> {
    @Override
    public int compare(Employee e1, Employee e2) {
        return Integer.compare(e1.getSalary(), e2.getSalary());
    }
}
```

[MainSort.java]

```
package sorting;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class MainSort {
    public static void main(String[] args) {
        List<Employee> listEmp = new ArrayList<>();
        listEmp.add(new Employee("Bobby", 3000));
        listEmp.add(new Employee("Erick", 1600));
```

```
        listEmp.add(new Employee("Rey", 2500));
        listEmp.add(new Employee("Anna", 3500));

        System.out.println("--- Data Awal ---");
        listEmp.forEach(System.out::println);

        // Sorting 1: Natural Order via Comparable (compareTo)
        Collections.sort(listEmp);
        System.out.println("\n--- Sorted by Name (Comparable / Natural Order) ---");
        listEmp.forEach(System.out::println);

        // Sorting 2: Custom Order via Comparator (compare)
        Collections.sort(listEmp, new SalaryComparator());
        System.out.println("\n--- Sorted by Salary (Comparator / Custom Order) ---");
        listEmp.forEach(System.out::println);
    }
}
```

Output:

```
--- exec:3.6.3:exec (default-cli) @ ProjectModul09 ---
--- Data Awal ---
Employee[name=Bobby , salary=3000]
Employee[name=Erick , salary=1600]
Employee[name=Rey   , salary=2500]
Employee[name=Anna  , salary=3500]

--- Sorted by Name (Comparable / Natural Order) ---
Employee[name=Anna  , salary=3500]
Employee[name=Bobby , salary=3000]
Employee[name=Erick , salary=1600]
Employee[name=Rey   , salary=2500]

--- Sorted by Salary (Comparator / Custom Order) ---
Employee[name=Erick , salary=1600]
Employee[name=Rey   , salary=2500]
Employee[name=Bobby , salary=3000]
Employee[name=Anna  , salary=3500]
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

5. Filtering

[Mahasiswa.java]

```
package filtering;

public class Mahasiswa {
    private String nim;
    private String nama;
    private double ipk;

    public Mahasiswa(String nim, String nama, double ipk) {
        this.nim = nim;
        this.nama = nama;
        this.ipk = ipk;
    }

    public String getNim() {
        return nim;
    }

    public String getNama() {
        return nama;
    }

    public double getIpk() {
        return ipk;
    }

    @Override
    public String toString() {
        return String.format("Mhs[NIM=%s, Nama=%-6s, IPK=%.2f]", nim, nama,
ipk);
    }
}
```

[MainFilter .java]

```
package filtering;

import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

public class MainFilter {
    public static void main(String[] args) {
        List<Mahasiswa> list = new ArrayList<>();
        list.add(new Mahasiswa("001", "Bobby", 3.25));
```

```
        list.add(new Mahasiswa("002", "Erick", 2.80));
        list.add(new Mahasiswa("003", "Rey", 3.75));
        list.add(new Mahasiswa("004", "Anna", 3.90));

        System.out.println("=== 1. FILTERING (IPK >= 3.50) ===");

        // --- Pendekatan 1: Imperatif (Konvensional) ---
        System.out.println("Cara Konvensional:");
        List<Mahasiswa> mhsCumlaudeKonven = new ArrayList<>();
        for (Mahasiswa m : list) {
            if (m.getIpk() >= 3.50) {
                mhsCumlaudeKonven.add(m);
            }
        }
        for (Mahasiswa m : mhsCumlaudeKonven) {
            System.out.println(m);
        }

        // --- Pendekatan 2: Deklaratif (Stream API) ---
        System.out.println("\nCara Stream API:");
        List<Mahasiswa> mhsCumlaudeStream = list.stream()
                .filter(m -> m.getIpk() >= 3.50)
                .collect(Collectors.toList());
        mhsCumlaudeStream.forEach(System.out::println);

        System.out.println("\n=== 2. SEARCHING / PENCARIAN TUNGGAL
===");

        // --- Pencarian Konvensional ---
        System.out.println("Cara Konvensional (Cari 'Erick'):");
        Mahasiswa targetKonven = null;
        for (Mahasiswa m : list) {
            if (m.getNama().equalsIgnoreCase("Erick")) {
                targetKonven = m;
                break; // Hentikan loop saat ditemukan
            }
        }
        System.out.println(targetKonven != null ? targetKonven : "Data tidak
ditemukan");

        // --- Pencarian via Stream API ---
        System.out.println("\nCara Stream API (Cari 'Erick'):");
        Mahasiswa targetStream = list.stream()
                .filter(m -> m.getNama().equalsIgnoreCase("Erick"))
                .findFirst()
                .orElse(null);
```

```
        System.out.println(targetStream != null ? targetStream : "Data tidak
ditemukan");
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ ProjectModul09 ---
=== 1. FILTERING (IPK >= 3.50) ===
Cara Konvensional:
Mhs[NIM=003, Nama=Rey   , IPK=3,75]
Mhs[NIM=004, Nama=Anna  , IPK=3,90]

Cara Stream API:
Mhs[NIM=003, Nama=Rey   , IPK=3,75]
Mhs[NIM=004, Nama=Anna  , IPK=3,90]

=== 2. SEARCHING / PENCARIAN TUNGGAL ===
Cara Konvensional (Cari 'Erick'):
Mhs[NIM=002, Nama=Erick , IPK=2,80]

Cara Stream API (Cari 'Erick'):
Mhs[NIM=002, Nama=Erick , IPK=2,80]
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

C. Unguided

Buatlah program Java untuk mengelola catatan hasil eksperimen model machine learning dengan ketentuan berikut:

1. Buat kelas untuk menyimpan hasil eksperimen dengan atribut nama model (String), akurasi (double, 0 sampai 1), dan waktu training dalam detik (int), lengkap dengan constructor, getter, dan toString() berformat Model=<nama>, akurasi=<akurasi>, waktu=<waktu> detik.
2. Tambahkan static variable untuk menghitung jumlah eksperimen yang dibuat, beserta static method untuk mengambil nilainya.
3. Urutkan secara default berdasarkan akurasi tertinggi ke terendah menggunakan Comparable.
4. Buat kelas terpisah yang mengimplementasikan Comparator untuk mengurutkan berdasarkan waktu training tercepat.
5. Di main (kelas terpisah), simpan data berikut ke dalam List dengan generics:

Nama Model	Akurasi	Waktu(detik)
Logistic Regression	0.82	12
Random Forest	0.91	45
SVM	0.87	60
KNN	0.79	5
Decision Tree	0.84	8

6. Tampilkan secara berurutan: jumlah eksperimen (static method), daftar urut akurasi tertinggi (Comparable), daftar urut waktu tercepat (Comparator), model dengan akurasi minimal 0.85 (lambda), dan pencarian model "KNN" (lambda, tampilkan null jika tidak ditemukan).

   Jawaban :

   [HasilEksperimen.java]

```
package Unguided;


public class HasilEksperimen implements Comparable<HasilEksperimen> {
    private String namaModel;
    private double akurasi;
    private int waktuTraining;


    // static variable: dipakai bersama oleh semua objek
    private static int jumlahEksperimen = 0;


    public HasilEksperimen(String namaModel, double akurasi, int waktuTraining)
{
        this.namaModel = namaModel;
        this.akurasi = akurasi;
        this.waktuTraining = waktuTraining;
```

```
    jumlahEksperimen++; // bertambah tiap objek baru dibuat
}

public String getNamaModel() {
    return namaModel;
}

public double getAkurasi() {
    return akurasi;
}

public int getWaktuTraining() {
    return waktuTraining;
}

// static method untuk mengambil jumlah eksperimen
public static int getJumlahEksperimen() {
    return jumlahEksperimen;
}

// urutan default: akurasi tertinggi ke terendah
@Override
public int compareTo(HasilEksperimen other) {
    return Double.compare(other.akurasi, this.akurasi);
}
```

```
    @Override
    public String toString() {
        return "Model=" + namaModel + ", akurasi=" + akurasi + ", waktu=" +
waktuTraining + " detik";
    }
}
```

[WaktuComparator.java]

```
package Unguided;

import java.util.Comparator;

public class WaktuComparator implements Comparator<HasilEksperimen> {
    // urutan: waktu training tercepat ke terlama
    @Override
    public int compare(HasilEksperimen e1, HasilEksperimen e2) {
        return Integer.compare(e1.getWaktuTraining(), e2.getWaktuTraining());
    }
}
```

[Main.java]

```
package Unguided;

import java.util.*;
import java.util.stream.Collectors;

public class Main {
    public static void main(String[] args) {
        List<HasilEksperimen> list = new ArrayList<>();
        list.add(new HasilEksperimen("Logistic Regression", 0.82, 12));
        list.add(new HasilEksperimen("Random Forest", 0.91, 45));
        list.add(new HasilEksperimen("SVM", 0.87, 60));
        list.add(new HasilEksperimen("KNN", 0.79, 5));
        list.add(new HasilEksperimen("Decision Tree", 0.84, 8));

        // 1. Jumlah eksperimen (static method)
        System.out.println("Jumlah eksperimen: " +
HasilEksperimen.getJumlahEksperimen());

        // 2. Sorting default (Comparable) -> akurasi tertinggi
```

```
        Collections.sort(list);
        System.out.println("Sorted by akurasi");
        list.forEach(System.out::println);

        // 3. Sorting Comparator -> waktu training tercepat
        Collections.sort(list, new WaktuComparator());
        System.out.println("Sorted by waktu training");
        list.forEach(System.out::println);

        // 4. Filtering lambda -> akurasi minimal 0.85
        System.out.println("Model dengan akurasi minimal 0.85");
        List<HasilEksperimen> hasilFilter = list.stream()
                .filter(e -> e.getAkurasi() >= 0.85)
                .sorted() // urut akurasi tertinggi agar sesuai contoh output
                .collect(Collectors.toList());
        hasilFilter.forEach(System.out::println);

        // 5. Pencarian lambda -> model KNN
        System.out.println("Pencarian model KNN");
        System.out.println(list.stream()
                .filter(e -> e.getNamaModel().equals("KNN"))
                .findFirst()
                .orElse(null));
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ ProjectModul09 ---
Jumlah eksperimen: 5
Sorted by akurasi
Model=Random Forest, akurasi=0.91, waktu=45 detik
Model=SVM, akurasi=0.87, waktu=60 detik
Model=Decision Tree, akurasi=0.84, waktu=8 detik
Model=Logistic Regression, akurasi=0.82, waktu=12 detik
Model=KNN, akurasi=0.79, waktu=5 detik
Sorted by waktu training
Model=KNN, akurasi=0.79, waktu=5 detik
Model=Decision Tree, akurasi=0.84, waktu=8 detik
Model=Logistic Regression, akurasi=0.82, waktu=12 detik
Model=Random Forest, akurasi=0.91, waktu=45 detik
Model=SVM, akurasi=0.87, waktu=60 detik
Model dengan akurasi minimal 0.85
Model=Random Forest, akurasi=0.91, waktu=45 detik
Model=SVM, akurasi=0.87, waktu=60 detik
Pencarian model KNN
Model=KNN, akurasi=0.79, waktu=5 detik
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

Penjelasan :

HasilEksperimen nyimpen nama model, akurasi, dan waktu training, lengkap dengan constructor, getter, dan toString() sesuai format soal. Ada variabel static jumlahEksperimen yang naik tiap objek dibuat dan dipakai bersama semua objek, dengan static method getJumlahEksperimen() buat ngambil nilainya. Kelas ini juga implements Comparable, dan compareTo()-nya sengaja membandingkan other dengan this (dibalik) supaya urutan defaultnya akurasi tertinggi ke terendah.

WaktuComparator adalah kelas terpisah yang implements Comparator dan meng-override compare() untuk membandingkan waktu training, jadi urutannya dari yang tercepat ke terlama. Keuntungannya, kita bisa bikin urutan lain tanpa mengubah kelas aslinya.

Main nyimpen kelima data di List<HasilEksperimen> (generics), lalu menampilkan jumlah eksperimen dari static method. Setelah itu list diurutkan pakai Collections.sort(list) (Comparable) dan Collections.sort(list, new WaktuComparator()) (Comparator). Filtering akurasi minimal 0.85 dan pencarian "KNN" memakai stream dengan lambda filter(). Pencarian memakai findFirst().orElse(null) supaya hasilnya null kalau model nggak ketemu.

D. Kesimpulan

Dari praktikum Modul 9 ini, program pencatatan hasil eksperimen model machine learning berhasil dibuat dengan menerapkan konsep static, collection, sorting, dan filtering. Static variable dan static method terbukti berguna buat menghitung jumlah objek yang dibuat, karena nilainya dipakai bersama oleh semua objek dan bisa diakses tanpa membuat instance. Collection dengan generics (List<HasilEksperimen>) memudahkan penyimpanan banyak objek secara aman karena tipenya terbatas.

Untuk sorting, Comparable cocok dipakai sebagai urutan default pada satu properti (akurasi tertinggi), sedangkan Comparator fleksibel buat urutan alternatif (waktu training tercepat) tanpa mengubah kelas aslinya. Sementara itu, ekspresi lambda dengan Stream API bikin proses filtering dan pencarian jadi lebih ringkas dibanding perulangan biasa. Hasilnya, program bisa menampilkan semua output sesuai contoh pada soal, termasuk null saat data yang dicari tidak ditemukan.